

ELE2 – LPG3

Eletiva 2 – Linguagem de Programação 3 (Java)
Prof. Davi Reis

Aula 02 – Orientação a Objetos – Revisão Básica

1



3

Conceitos

- Classe
 - Objeto
 - Instância
- Construtor e Destrutor
- Encapsulamento (membros)
 - Atributos
 - Métodos
- Herança
- Polimorfismo

4

Classe

- Abstrai um conjunto de objetos/entidades com características e comportamentos similares entre si.
- Define (ou descreve) a estrutura e o comportamento que os objetos compartilharão.
- Para isso, dispõe de:
 - Atributos: características ou estados possíveis destes objetos.
 - Métodos: ações que o objeto pode realizar.

5

Estrutura de um Classe

- Atributos
- Métodos
- Construtores
- Destrutor

▪ Sintaxe:

```
class NomeDaClasse{
    modificador tipo atributo
    modificador tipo método (tipo parâmetro1, tipo parâmetro2..., tipo parâmetroN){
        //Código...
    }
}
```

Prof. Davi

ELE2

6

6

Atributos

- São declarações das classes que representam características de instância que possam assumir estados.
- No exemplo, **dia**, **mes** e **ano** são atributos do tipo inteiro da classe **Data**, enquanto **hora**, **minuto** e **segundo** são atributos do tipo inteiro da classe **Horario**. Na classe Principal, nenhum atributo foi declarado.

```
1 package aularevisao;
2
3 class Data{
4     int dia, mes, ano;
5 }
6 class Horario{
7     int hora, minuto, segundo;
8 }
9 public class Principal {
10     public static void main(String[] args) {
11         Data data = new Data();
12         data.ano = 2025; data.mes = 12; data.dia = 31;
13
14         Horario horario = new Horario();
15         horario.hora = 23; horario.minuto = 59; horario.segundo = 59;
16
17         System.out.println(String.format("Data: %d/%d/%d",
18             data.dia, data.mes, data.ano));
19
20         System.out.println(String.format("Hora: %d:%d:%d",
21             horario.hora, horario.minuto, horario.segundo));
22     }
23 }
```

Saida - AulaRevisao (run)

```
run:
Data: 31/12/2025
Hora: 23:59:59
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

Prof. Davi

ELE2

7

7

Métodos

- São declarações das classes que representam ações que a mesma pode executar em função dos valores de seus atributos.
- No exemplo, um método chamado **retornarInfo()** em cada classe retorna uma String representando a data e a hora em função dos valores dos atributos de suas respectivas instâncias.

```
1 package aularevisao;
2
3 class Data{
4     int dia, mes, ano;
5     String retornarInfo(){
6         return String.format("Data: %d/%d/%d", dia, mes, ano);
7     }
8 }
9 class Horario{
10     int hora, minuto, segundo;
11     String retornarInfo(){
12         return String.format("Hora: %d:%d:%d", hora, minuto, segundo);
13     }
14 }
15 public class Principal {
16     public static void main(String[] args) {
17         Data data = new Data();
18         data.ano = 2025; data.mes = 12; data.dia = 31;
19
20         Horario horario = new Horario();
21         horario.hora = 23; horario.minuto = 59; horario.segundo = 59;
22
23         System.out.println(data.retornarInfo());
24         System.out.println(horario.retornarInfo());
25     }
26 }
```

Salida - AulaRevisao (run)

run:

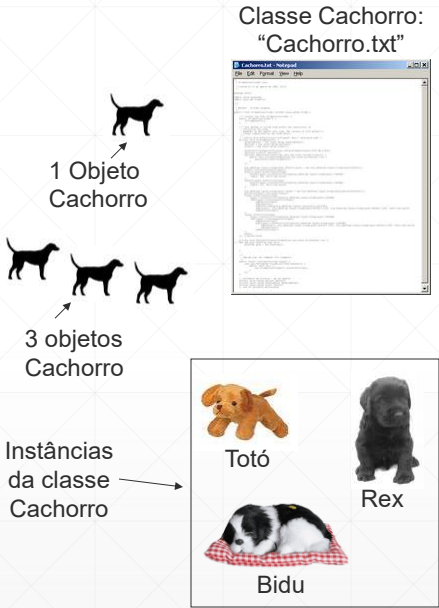
Data: 31/12/2025

Hora: 23:59:59

CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)

Objetos e Instâncias

- Objeto é uma entidade que pode ser física, conceitual ou de software. Ou seja, uma representação genérica.
- Instância é usada com o sentido de exemplo. É a concretização da classe. Ou seja, são os objetos de fato criados e ocupando espaço na memória.



Construtor

- Método especial. É disparado automaticamente na instancição da classe.
- Utilizado basicamente para inicializar os estados dos atributos.
- Deve possuir sempre o mesmo nome da classe.
- Pode ser do tipo **default** ou **não-default**.
- Retorna sempre a instância da classe.
- Normalmente **public**, mas ter outros modificadores de acesso.
- No exemplo a seguir, foram criados dois construtores. Cada um deles é chamado de acordo com a criação da instância, pela lista de argumentos.

Prof. Davi

ELE2

10

10

Construtor Exemplo:

```

1 package aularevisao;
2
3 class Data{
4     int dia, mes, ano;
5     public Data() {
6         dia = mes = ano = 0;
7     }
8     public Data(int dia, int mes, int ano) {
9         this.dia = dia; this.mes = mes; this.ano = ano;
10    }
11    String retornarInfo(){
12        return String.format("Data: %d/%d/%d", dia, mes, ano);
13    }
14 }
15
16 public class Principal {
17     public static void main(String[] args) {
18         Data data = new Data();
19         data.ano = 2025; data.mes = 12; data.dia = 31;
20         System.out.println(data.retornarInfo());
21
22         Data data2 = new Data(31,12,2029);
23         System.out.println(data2.retornarInfo());
24     }
25 }
26
27 : Saída - AulaRevisao (run)
28
29 Data: 31/12/2025
30 Data: 31/12/2029

```

Prof. Davi

ELE2

11

11

Destrutor

- O método Destrutor não pode ser sobrecarregado porque não aceita parâmetros e é executado quando o objeto é excluído.

```
obj.finalize();
```

- Não é recomendado para a liberação de recursos, pois o momento em que ele é chamado pelo *garbage collector* pode ser imprevisível.
- Para liberação de recursos, é aconselhável utilizar como “try-finally” ou implementar interfaces como “AutoCloseable”, se necessário.

Prof. Davi

ELE2

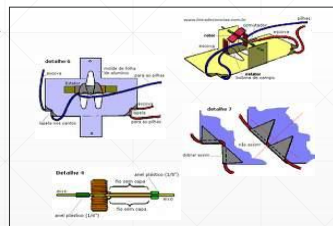
12

12

Encapsulamento

- Mecanismo que permite separar detalhes de funcionamento – características (atributos) e funções (métodos) – de sua interface.
- Para isso, limita o acesso aos membros da classe
 - Exemplo: Para utilizarmos um liquidificador, não precisamos saber detalhes de seu funcionamento. A única interface que conhecemos são seus botões.

Classe
Liquidificador



Instância de um objeto de liquidificador: muito mais fácil de usar!

Prof. Davi

ELE2

13

13

Encapsulamento

- Utiliza-se de modificadores de acesso, tais como:
 - Public:** Acessível a partir de qualquer outra classe, independente do package.
 - Protected:** Acessível a partir de qualquer classe derivada, no mesmo package.
 - Private:** Acessível apenas dentro da própria classe.
 - Static:** Membro compartilhado com todas as instâncias da classe.
 - Final:** Constante da classe.
 - Friendly:** Acessível a partir de qualquer outra classe do mesmo package (também chamado de “default” ou “package-private”).

Prof. Davi

ELE2

14

14

Encapsulamento - Atributo

- Os atributos são características (variáveis) definidas em uma classe e podem ser classificados em dois tipos:
 - Atributos de Instância:** são propriedades que pertencem a cada objeto individualmente e armazenam o estado específico de cada instância.
 - Atributos de Classe:** são propriedades associadas à classe em si, cujo valor/estado é compartilhado por todas as instâncias dessa classe.

```

class CartaDeBaralho{
    public char simbolo;
    public image Naipes;
    public static ListaDeSimbolos[] = {0,1,2,3,4,5,6,7,8,9,J,Q,K,A};
    public static ListaDeNaipes[] = { ♥ , ♠ , ♦ , ♣ };
}
  
```

Atributos de Instância

Atributos de Classe

Prof. Davi

ELE2

15

15

Encapsulamento - Método

- Os métodos são rotinas executadas por um objeto ao receber uma mensagem, determinando seu comportamento. Eles são análogos às funções ou procedimentos da programação estruturada, mas estão associados a um contexto específico de um objeto ou classe.
- É uma rotina que é executada por um objeto ao receber uma mensagem. De certo modo, é a descrição da forma como o objeto realiza uma tarefa.
- Os métodos determinam o comportamento dos objetos de uma classe e são análogos às funções ou procedimentos da programação estruturada.
- Assim como os atributos, os métodos podem ser encapsulados por meio de modificadores de acesso, controlando como eles podem ser chamados dentro e fora da classe.

Prof. Davi

ELE2

16

16

Encapsulamento de atributos

Exemplo:

```

1 package aularevisao;
2 class Data{
3     private int dia, mes, ano;
4
5     public int getDia() { return dia; }
6     public void setDia(int dia) {
7         if (dia > 0 && dia < 32) this.dia = dia;
8     }
9     public int getMes() { return mes; }
10    public void setMes(int mes) {
11        if (mes > 0 && mes < 13) this.mes = mes;
12    }
13    public int getAno() { return ano; }
14    public void setAno(int ano) { this.ano = ano; }
15
16    String retornarInfo(){
17        return String.format("Data: %d/%d/%d", dia, mes, ano);
18    }
19 }
20
21 public class Principal {
22     public static void main(String[] args) {
23         Data data = new Data();
24         data.setDia(31); data.setMes(12); data.setAno(2025);
25         System.out.println(data.retornarInfo());
26     }
27 }

```

Saida - AulaRevisao (run)

```

run:
Data: 31/12/2025

```

Prof. Davi

ELE2

17

17

Herança - Definição

- Conhecida também como Generalização.
- Definição de uma classe a partir de outra, herdando os seus membros (não-privados).
- A classe herdeira é chamada subclasse.
- A classe herdada é chamada superclasse.
- Pode ser utilizado em cascata, criando hierarquias com várias gerações de classes.
- Permite que as classes compartilhem atributos e métodos baseados em um **relacionamento**.

Prof. Davi

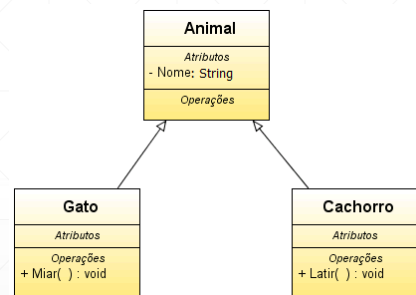
ELE2

18

18

Subclasse e superclasse

- Superclasse é a classe herdada. Ou seja, a classe Pai.
- Subclasse é a classe herdeira. Ou seja, a classe Filha.
- No exemplo, Animal é a superclasse.
- No exemplo, Cachorro e Gato são subclasses.



Prof. Davi

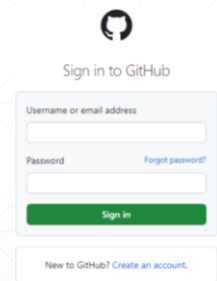
ELE2

19

19

Antes de prosseguir com código...

- Recomenda-se fortemente o uso e a familiarização com ferramentas de controle de versão, colaboração, integração etc., tais como o Git (e GitHub), pois é algo praticamente obrigatório no mercado de trabalho.
- Por isso, mas não somente, é importante aprender a utilizá-la e criar um portfólio com todos os seus códigos e projetos que estiver estudando ou desenvolvendo.
- Para auxiliar, segue um curso simples e gratuito:
<https://www.udemy.com/course/git-e-github-para-iniciantes/>



Obrigado!